Formulário de Cadastro

HTML + CSS + JS + Supabase + GitHub + Vercel

Desenvolvimento Front-End para Web — Unidade Integradora

Prof. Marcos Venícios — Cruzeiro do Sul

O que vamos construir hoje?

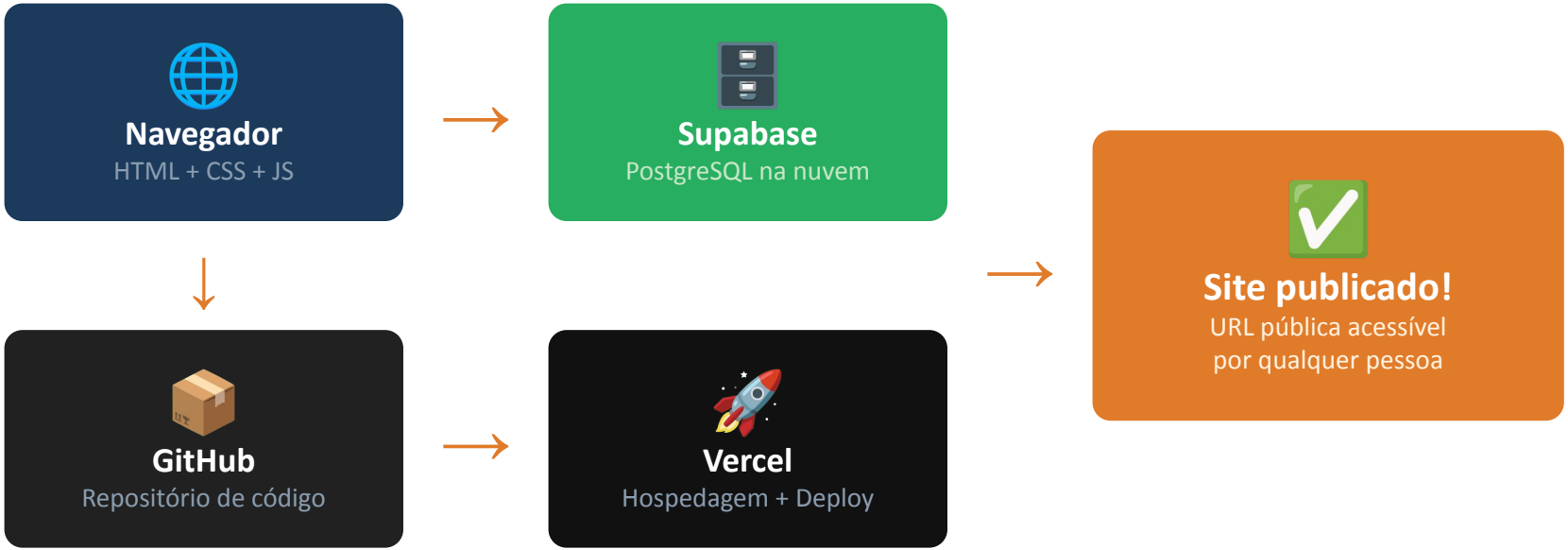
Objetivo

Um formulário de cadastro da TechStore, publicado na internet, com dados salvos em banco de dados real

Stack Tecnológica

📄 **HTML**	Estrutura do formulário	🎨 **CSS**	Visual moderno	⚡ **JavaScript**	Lógica e integração
🗄️ **Supabase**	Banco de dados na nuvem	📦 **GitHub**	Controle de versão	🚀 **Vercel**	Publicação automática

Arquitetura da Aplicação



Roteiro da Aula

1	**Supabase**	Criar projeto e tabela	**20 min**
2	**HTML**	Estrutura do formulário	**20 min**
3	**CSS**	Estilização visual	**15 min**
4	**JavaScript**	Lógica + integração	**25 min**
5	**GitHub**	Repositório e push	**20 min**
6	**Vercel**	Deploy automático	**15 min**
7	**Teste**	Validação cruzada	**10 min**

Total: ~125 min + 25 min reserva

Bloco 1 — Supabase

Criando o banco de dados na nuvem

O que é o Supabase?

Backend-as-a-Service (BaaS)
Um banco de dados PostgreSQL completo na nuvem, com API pronta para usar.
Sem precisar instalar nada.

- 🆓 Gratuito (plano Free)
- ⚡ API REST automática
- 🔒 Row Level Security
- 📊 Painel visual dos dados

Analogia

❌ **Antes**
Instalar MySQL, configurar servidor, escrever API manualmente, hospedar...

✅ **Com Supabase**
Criar conta, criar tabela pelo painel, chamar a API do front-end. Pronto!

Configurando o Supabase

1

Criar conta

Acesse supabase.com → Sign Up (use GitHub)

2

Novo projeto

New Project → Nome: "techstore" → Senha do banco → Região: São Paulo

3

Criar tabela

Table Editor → New Table → Nome: "cadastros"

4

Definir colunas

nome (text), email (text), telefone (text), created_at (timestamptz)

5

Copiar credenciais

Settings → API → Project URL + anon public key

SQL — Criando a Tabela

```
CREATE TABLE cadastros (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome TEXT NOT NULL,
  email TEXT NOT NULL,
  telefone TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

id — Identificador único, gerado automaticamente
NOT NULL — Campo obrigatório (não aceita vazio)
DEFAULT NOW() — Data/hora automática no momento do cadastro

Row Level Security (RLS)

⚠️ **Atenção!**
O Supabase bloqueia TUDO por padrão. Sem a política abaixo, o formulário vai retornar erro ao tentar salvar os dados.

```
-- Habilitar RLS na tabela
ALTER TABLE cadastros ENABLE ROW LEVEL SECURITY;

-- Permitir que qualquer pessoa insira dados
CREATE POLICY "Permitir insercao anonima"
  ON cadastros FOR INSERT TO anon
  WITH CHECK (true);
```

Em produção, as políticas seriam mais restritivas. Para a aula, permitimos inserção anônima.

Bloco 2 — HTML

Estrutura semântica do formulário

Anatomia de um Formulário HTML

<form>

Container que agrupa todos os campos

<label>

Rótulo acessível — conecta ao input via "for"



<input>

Campo de entrada — type define o comportamento

<button>

Botão de ação (type="submit")

Tipos de Input e Validação Nativa

Tipo	Comportamento	Uso comum	Validação
type="text"	Texto livre	Nome, endereço	—
type="email"	Valida @ e domínio	E-mail	Formato automático
type="tel"	Teclado numérico (mobile)	Telefone	Sem validação
type="password"	Oculta caracteres	Senhas	Pontos/asteriscos
type="number"	Só aceita números	Idade, CEP	Setas +/−

Atributos de validação do HTML5

`required` — campo obrigatório `placeholder` — texto-dica `pattern` — regex personalizada

Código HTML do Formulário

```
<form id="formCadastro">

  <label for="nome">Nome Completo *</label>
  <input type="text" id="nome" name="nome"
         placeholder="Ex: Maria da Silva" required>

  <label for="email">E-mail *</label>
  <input type="email" id="email" name="email"
         placeholder="Ex: maria@email.com" required>

  <label for="telefone">Telefone</label>
  <input type="tel" id="telefone" name="telefone"
         placeholder="Ex: (11) 99999-9999">

  <button type="submit">Cadastrar</button>

</form>
```

Bloco 3 — CSS

Estilização moderna do formulário

Variáveis CSS — Cores Centralizadas

```
:root {
  --cor-primaria: #1B3A5C;
  --cor-secundaria: #E07B2A;
  --cor-fundo: #F5F0E8;
  --cor-branco: #FFFFFF;
  --cor-texto: #212121;
  --cor-sucesso: #27AE60;
  --cor-erro: #E74C3C;
}
```



💡 **Vantagem:** Para mudar o visual inteiro, basta alterar os valores no :root

Conceitos-Chave do CSS Aplicado

📦 Flexbox

```
display: flex
justify-content: center
align-items: center
```

Centraliza o card na tela

✨ Transições

```
transition: all 0.3s ease
```

Animações suaves em hover/focus

🎯 Pseudo-classes

```
:focus  :hover  :active
::placeholder
```

Estilos por estado do elemento

📱 @media

```
@media (max-width: 480px)
```

Adapta layout para mobile

Bloco 4 — JavaScript



Lógica, validação e integração com Supabase

Fluxo do Envio



✅ Sucesso

Mensagem verde
Formulário limpa (reset)
Dados no banco

❌ Erro

Mensagem vermelha
Botão reabilitado
Console com detalhes

Conceitos JavaScript Utilizados

`document.getElementById()`	Busca um elemento HTML pelo atributo id
`addEventListener("submit", fn)`	Escuta o evento de envio do formulário
`evento.preventDefault()`	Impede o recarregamento da página
`.value.trim()`	Pega o texto digitado e remove espaços extras
`async / await`	Espera operações demoradas (rede) terminarem
`try / catch`	Tenta executar; se falhar, trata o erro

JavaScript — Conexão com Supabase

```
// Carregue via CDN no HTML:
// <script src="https://cdn.jsdelivr.net/npm/
//   @supabase/supabase-js@2"></script>

const SUPABASE_URL = 'https://SEU-PROJETO.supabase.co';
const SUPABASE_ANON_KEY = 'SUA-CHAVE-AQUI';

const supabase = window.supabase
  .createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
```

⚠️ **Cada aluno usa SUAS credenciais!**

A anon key é **pública** — pode ficar no código do front-end.
A segurança é feita pelas policies de RLS no Supabase (Bloco 1).

JavaScript — Enviando Dados

```
formulario.addEventListener('submit', async (evento) => {
  evento.preventDefault();

  const nome = document.getElementById('nome')
    .value.trim();
  const email = document.getElementById('email')
    .value.trim();
  const telefone = document.getElementById('telefone')
    .value.trim();

  const { data, error } = await supabase
    .from('cadastros')
    .insert([{ nome, email, telefone }])
    .select();

  if (error) {
    mostrarMensagem('Erro: ' + error.message, 'erro');
  } else {
    mostrarMensagem('Cadastro realizado!', 'sucesso');
```

Bloco 5 — GitHub

Versionamento e repositório remoto

Git e GitHub em 60 Segundos

📁 Git

Sistema de controle de versão.
Registra cada mudança no código.
Como um "histórico de salvamentos".

🌐 GitHub

Plataforma online para armazenar repositórios Git. Como um "Google Drive para código".

Fluxo simplificado

Salvar arquivo (index.html)

git add . (prepara)

git commit (registra)

git push (envia)

Alternativa Simples — Upload pelo GitHub

1

Criar conta no GitHub

github.com → Sign Up (gratuito)

2

Criar repositório

New Repository → Nome: "techstore-cadastro" → Public → Create

3

Upload do arquivo

Add file → Upload files → Arrastar o index.html → Commit changes

💡 Não precisa instalar o Git! O upload manual pelo site do GitHub funciona perfeitamente para esta aula.

Bloco 6 — Vercel

Publicação automática na internet

O que é a Vercel?

Plataforma de deploy automático

Conecta ao GitHub e publica automaticamente a cada push.
Cada projeto recebe uma URL pública gratuita (HTTPS incluído).



Conectar

Login com GitHub



Importar

Selecionar repositório



Deploy

Automático (10 segundos)



URL

seusite.vercel.app

Deploy na Vercel — Passo a Passo

1. Acesse vercel.com e clique "Sign Up"
2. Faça login com sua conta do GitHub
3. Clique "Add New..." → "Project"
4. Selecione o repositório "techstore-cadastro"
5. Clique "Deploy" (sem configurar nada!)
6. Aguarde ~10 segundos → URL pronta!

🎉 **Resultado: https://techstore-cadastro-seunome.vercel.app**

Bloco 7 — Teste Cruzado

Validação real entre os alunos

Dinâmica: Teste Cruzado

Compartilhe sua URL da Vercel com o colega ao lado



Preencha o formulário do colega com dados reais

Volte ao SEU Supabase e verifique se os dados chegaram

Teste também no celular — funciona? É responsivo?

Dê feedback: o que achou do visual? Funcionou tudo?

O que aprendemos hoje

	HTML	Formulários semânticos, validação nativa
	CSS	Variáveis, estados, responsividade
⚡	**JS**	Eventos, async/await, integração com API
	Supabase	Banco na nuvem, SQL, RLS
	GitHub	Repositório, versionamento
	Vercel	Deploy contínuo, URL pública

🎉 **Vocês publicaram uma aplicação web completa na internet!**



TechStore Cadastro

Formulário Full-Stack com Supabase + GitHub + Vercel

Prof. Marcos Venícios
Desenvolvimento Front-End para Web
Universidade Cruzeiro do Sul — 2026/2